Exactly-Once Filesystem Operations in xv6, with a Crash-Resilient AI Agent

Repository: `https://github.com/Sam-X-code/NASA` (final integration commit 9c5667d)

1 Introduction

A filesystem operation can commit successfully even when the caller crashes or loses the response before learning that it completed. For non-idempotent operations this makes retrying unsafe. If a file contains A and the caller appends B, the kernel may commit the append and then fail before replying. On restart the caller cannot tell whether the append happened. Retrying can produce ABB, while not retrying may lose the B.

xv6 uses a write-ahead log (WAL) to keep the filesystem consistent across crashes, but the log does not record which logical request a transaction belonged to. This project adds that information. Our main deliverable is one new system call, `effect()`, which lets a caller safely retry selected filesystem mutations. As an extension, we built a host-side AI agent that uses `effect()` and can recover an interrupted multi-step task.

2 Core Contribution: the effect() System Call

`effect()` takes a 64-bit operation key chosen by the caller, plus an operation and its arguments. Create and append are currently supported. The kernel looks the key up in a persistent Effect Persistence Table (EPT). If the key exists, the kernel returns the stored result and does not repeat the mutation. If it does not exist, the kernel performs the mutation and inserts the key and result into the EPT inside the same WAL transaction.

Because both changes belong to one transaction, a crash before the commit record is written exposes neither the mutation nor the EPT entry, and a crash after the commit exposes both. A retry can therefore tell a committed operation from one that did not commit.

The EPT uses 10 disk blocks of 512 bytes starting at block 59. Each entry is 16 bytes (a 64-bit key and a 64-bit result), giving room for 320 records. Lookup uses hashing with linear probing. A global effect lock serializes effect operations so lookup and insertion cannot race.

Each call proceeds as follows: acquire the effect lock; validate the operation and arguments; look up the key; if found, return the stored result; otherwise begin a transaction, perform the mutation, insert the key and result, end the transaction, and return the result.

The kernel only accepts four paths (`/report.txt`, `/inbox.txt`, `/important.txt`, `/agentlog.txt`). Append payloads must be between 1 and 20 bytes, contain no NUL bytes, and be valid UTF-8. These checks run before the EPT is touched.

3 Extension: AI Agent with a Durable Task Journal

The Python agent in `agent_lab/` drives xv6 running in QEMU. It separates planning from execution: a planner proposes an action, a validation step checks it, and only approved actions are sent to the guest. Three planners are provided. `RulePlanner` gives deterministic behaviour for repeatable tests. `InjectedPlanner` supplies malicious or suspicious instructions to test the validation and security behaviour. `OllamaPlanner` uses a locally hosted language model. An optional model-based assessor can label actions as safe, suspicious or uncertain, but it is advisory only and never grants permission.

The agent keeps a task journal as a JSON-lines file. Before an action is sent to the guest, the agent records a `PREPARED` entry containing the task, the step, the original action and arguments, and a stable effect key. The step then moves to `RUNNING` and finally `COMPLETED`. On startup the agent loads the journal and recovers any pending step by replaying the original action with the original key. It does not ask the planner for a new action. If the journal is malformed or recovery cannot finish safely, the agent stops instead of starting new work.

The journal and the EPT cover different things. The journal remembers what the agent intended to do and how far it got. The EPT remembers which effects have actually committed. Together they cover the critical crash window: the guest commits an append, but the agent crashes before recording `COMPLETED`. After restart the agent resends the same key, the kernel finds it in the EPT, and the stored result is returned without a second append. If the original transaction had not committed, the key is absent and the operation runs normally.

4 Security and Authorization

We also added process-aware security controls. Each process has its own security context. Reading non-empty data from an untrusted file, or executing an untrusted program, taints the process, and certain operations are restricted for tainted processes. File trust is stored in the existing `dinode.major` field, so the on-disk inode layout is unchanged.

Direct `effect()` calls from guest processes are denied unless the process holds the effect capability, which is granted by running the trusted `/effectctl` program. The capability is not inherited through `fork` and is reset when a different program is executed. This is a shared guest-side capability, not cryptographic authentication of one specific agent. The kernel check is the enforcement point; the host agent's validation and the optional assessor are additional layers.

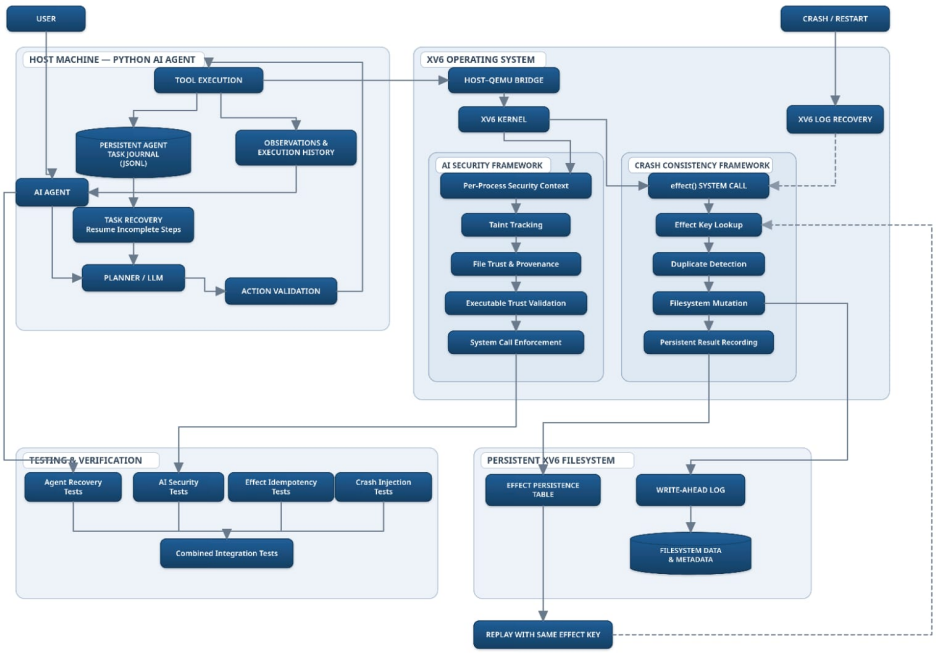


Figure 1: Project architecture.

5 Evaluation

The main end-to-end test used a real QEMU instance. The file began containing A, and the agent appended B with a stable key. After the effect committed, we forcibly killed the QEMU process and restarted it on the same disk image. The file read AB before the kill, AB after the restart and before replay, and AB after the agent replayed the pending step. The original EPT entry was still present, no second B was appended, and the WAL was clean.

Other results: 25 Python unit tests passed; the EPT functional suite passed 10 of 10; the effect authorization tests, including the untrusted-caller test, passed; the kernel security suite had 0 failures; the host-agent crash and replay demonstration ended with AB; and `git diff -check` passed.

These results support the claim that the tested EPT path, authorization checks and recovery path work as designed. They do not establish performance, behaviour at full EPT capacity, multi-agent safety, resistance to disk corruption, or durability under host power loss.

6 Challenges

Integrating the EPT with the WAL required treating the log header as the commit point and making sure the mutation and the EPT record were logged together. Lock ordering had to account for the effect lock, filesystem transactions, inode locks and security locks. Per-process taint state had to be handled correctly across fork, exec and exit. Trust metadata had to fit into the existing inode layout. The host journal had to store the original action and key so recovery would not depend on a model reproducing the same request. Finally, we merged separately developed Git histories into the final integration commit.

7 Limitations and Future Work

The guarantee applies only to supported operations sent through `effect()`, with the same key and identical arguments on every retry; ordinary syscalls do not gain it. The EPT stores no payload fingerprint and holds at most 320 records, with no reclamation policy yet. A single global lock limits throughput. The authorization capability is shared rather than per-agent. The journal assumes a single active writer, and its power-loss durability has not been verified. Our tests cover software and QEMU crashes, not hardware faults.

Future work includes measuring latency and behaviour near capacity, adding safe EPT reclamation, attaching a request fingerprint to each key, finer-grained locking, stronger agent-to-guest authentication, testing concurrent agents, and verifying journal flush behaviour.

8 Conclusion

We added an operation-level exactly-once mechanism to xv6 by committing a key and result record in the same WAL transaction as the filesystem mutation. A host-side AI agent with a durable journal uses it to recover interrupted tasks and safely retry uncertain operations. The QEMU kill-and-restart test showed the file staying AB throughout, with no duplicate append. The guarantees are scoped as described in Section 7.